Kegiatan Belajar 2.3: DOM, Event, dan Debugging

Hari 7 · 8 JP · UK 1.5, 1.1, dan 1.2

Tujuan KB 2.3. Setelah mengikuti KB ini, Anda mampu memilih dan mengubah elemen halaman dengan JavaScript, menanggapi event pengguna, dan menemukan serta memperbaiki kesalahan program dengan Console.

Kegiatan	JP
DOM: memilih dan mengubah elemen	2
Event: klik, input, submit	2
Praktik halaman latihan DOM	1
Membuat elemen dari data dan keamanan `innerHTML`	1
Jenis kesalahan dan debugging dengan Console	2
Jumlah	**8**

Materi 1: DOM

DOM (*Document Object Model*) adalah representasi halaman HTML sebagai **pohon objek** yang dapat dibaca dan diubah oleh JavaScript. Objek `document` adalah pintu masuknya.

Memilih elemen:

Perintah	Hasil
`document.getElementById("judul")`	Elemen dengan `id="judul"`
`document.querySelector(".card")`	Elemen **pertama** yang cocok dengan selector CSS
`document.querySelectorAll(".card")`	**Semua** elemen yang cocok (seperti array)

Jika elemen tidak ditemukan, hasilnya `null`, dan kode yang memakainya akan menghasilkan kesalahan.

Mengubah elemen:

Perintah	Fungsi
`elemen.textContent = "teks"`	Mengganti isi dengan **teks biasa** (aman)
`elemen.innerHTML = "<b>teks</b>"`	Mengganti isi dengan **HTML**
`elemen.classList.add("aktif")`	Menambah class CSS
`elemen.classList.remove("aktif")`	Menghapus class
`elemen.classList.toggle("aktif")`	Menambah jika belum ada, menghapus jika ada
`elemen.setAttribute("href", "#a")`	Mengubah atribut
`elemen.value`	Membaca atau mengisi nilai `<input>`

Untuk mengubah **tampilan**, lebih baik mengganti *class* (yang diatur di CSS) daripada mengubah gaya langsung lewat JavaScript. Dengan begitu pembagian tugas tetap rapi: HTML untuk struktur, CSS untuk tampilan, JavaScript untuk perilaku.

Materi 2: Event

Event adalah kejadian di halaman: klik, mengetik, mengirim formulir, dan lainnya. Pola dasarnya:

```
elemen.addEventListener("namaEvent", function (e) {
  // kode yang dijalankan saat event terjadi
});
```

Event	Terjadi saat
`click`	Elemen diklik
`input`	Isi kolom input berubah (setiap ketikan)
`submit`	Formulir dikirim
`keyup`	Tombol keyboard dilepas

Objek `e` (event) membawa informasi:

- `e.target`: elemen yang memicu event.
- `e.preventDefault()`: **mencegah perilaku bawaan**, misalnya formulir yang memuat ulang halaman saat dikirim.

Praktik 1: Halaman Latihan DOM

Buat `latihan/dom.html`:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Latihan DOM</title>
  <style>
    .aktif { color: crimson; }
  </style>
</head>
<body>
  <h1 id="judul">Latihan DOM</h1>
  <p id="pesan">Belum ada aksi.</p>

  <input type="text" id="nama" placeholder="Ketik nama Anda">
  <button id="tombol">Sapa</button>

  <h2>Daftar Kopi</h2>
  <ul id="daftar"></ul>

  <script src="../js/data.js"></script>
  <script src="dom.js"></script>
</body>
</html>
```

Buat `latihan/dom.js`:

```
// 1. Memilih elemen
const judul = document.getElementById("judul");
const pesan = document.getElementById("pesan");
const inputNama = document.getElementById("nama");
const tombol = document.getElementById("tombol");
const daftar = document.getElementById("daftar");

// 2. Event klik pada tombol: membaca input, mengubah teks
tombol.addEventListener("click", function () {
  const nama = inputNama.value.trim();

  if (nama === "") {
    pesan.textContent = "Nama belum diisi.";
  } else {
    pesan.textContent = "Halo, " + nama + "!";
  }
});

// 3. Event klik pada judul: mengganti class
judul.addEventListener("click", function () {
  judul.classList.toggle("aktif");
});

// 4. Membuat elemen dari data (array PRODUCTS di data.js)
PRODUCTS.forEach(function (p) {
  const li = document.createElement("li");
  li.textContent = p.nama + " (" + p.asal + ")";
  daftar.appendChild(li);
});
```

Uji: klik tombol dengan kolom kosong dan terisi; klik judul berkali-kali; periksa daftar kopi. Perhatikan bahwa `data.js` dimuat **sebelum** `dom.js`, karena `dom.js` memakai `PRODUCTS`.

Materi 3: Membuat Elemen dari Data dan Keamanan

Ada dua cara mengisi halaman dari data:

Cara	Contoh	Catatan
`createElement` + `appendChild`	Contoh di Praktik 1	Aman, tetapi panjang untuk struktur rumit
`innerHTML` dengan template literal	`daftar.innerHTML = PRODUCTS.map(...).join("")`	Ringkas untuk kartu; **wajib waspada**

Contoh dengan `innerHTML`:

```
daftar.innerHTML = PRODUCTS.map(function (p) {
  return `<li>${p.nama}</li>`;
}).join("");
```

`map` menghasilkan array. Ubah menjadi satu teks dengan `.join("")`. Tanpa `join`, tanda koma akan muncul di antara elemen.

Keamanan. `innerHTML` menafsirkan teks sebagai **HTML**. Jika data berasal dari pengguna (misalnya isi formulir) dan berisi tag berbahaya seperti `<img src=x onerror=...>`, kode itu ikut dijalankan. Serangan ini disebut **XSS** (*cross-site scripting*). Aturannya:

- Untuk teks biasa, pakai `textContent`.

- Jika harus memakai `innerHTML` dengan data yang tidak sepenuhnya Anda kendalikan, **bersihkan dulu** dengan fungsi `esc()` yang akan kita pakai di KB 2.4.

Materi 4: Jenis Kesalahan dan Debugging

Semua pemrogram membuat kesalahan. Yang membedakan adalah cara menemukannya. **Debugging** adalah proses menemukan dan memperbaiki kesalahan.

Jenis	Ciri	Contoh
Syntax error	Kode salah tulis; program **tidak berjalan sama sekali**	Kutip atau kurung tidak berpasangan
Runtime error	Program berjalan, lalu **berhenti** saat kode bermasalah dieksekusi	`ReferenceError` (nama tidak ada), `TypeError` (misalnya `null.value`)
Logic error	Program berjalan, **tetapi hasilnya salah**	Diskon 10% ditulis `total * 10`

Membaca pesan kesalahan di Console. Contoh:

```
Uncaught ReferenceError: totl is not defined
    at app.js:12
```

Artinya: nama `totl` tidak dikenal, di berkas `app.js` baris 12. Klik nama berkas di Console untuk langsung menuju barisnya.

Langkah debugging yang disarankan:

1. **Baca** pesan kesalahan (jenis, berkas, baris).
2. **Periksa** baris tersebut dan baris di sekitarnya (ejaan, tanda kurung, koma).
3. **Telusuri** nilai variabel dengan `console.log(...)`. Untuk array objek, `console.table(PRODUCTS)` menampilkan tabel yang rapi.
4. **Persempit** masalah: matikan sebagian kode (jadikan komentar) sampai bagian bermasalah ketemu.
5. **Perbaiki**, lalu **uji ulang**, termasuk pada nilai batas.

DevTools juga menyediakan tab **Sources** untuk memasang *breakpoint* (jeda di baris tertentu) dan memeriksa nilai variabel langkah demi langkah. Instruktur akan mendemonstrasikannya.

Praktik 2: Debugging Terpandu

Setiap potongan kode berikut memiliki kesalahan. Salin ke berkas latihan, jalankan, baca pesan di Console, lalu perbaiki. Catat **jenis kesalahan** dan **perbaikannya**.

Kode 1

```
const nama = "Rahma;
console.log("Halo " + nama);
```

Kode 2

```
let total = 0;
for (let i = 1; i <= 5; i++) {
  total = total + i;
}
console.log("Total: " + totl);
```

Kode 3

```
const produk = { nama: "Kopi Gayo", harga: 80000 };
console.log(produk.stok.toString());
```

Kode 4 (tidak ada pesan kesalahan, tetapi hasilnya salah; diskon seharusnya 10%)

```
function hitungDiskon(total) {
  if (total > 100000) {
    return total * 10;
  }
  return 0;
}
console.log(hitungDiskon(200000));
```

Kode 5 (halaman memiliki `<button id="tombol">`)

```
const tombol = document.getElementById("Tombol");
tombol.addEventListener("click", function () {
  console.log("Diklik");
});
```

Latihan 3

1. Pada `dom.html`, tambahkan tombol **"Reset"** yang mengembalikan pesan ke "Belum ada aksi." dan mengosongkan kolom nama.
2. Tampilkan **jumlah karakter** yang diketik di kolom nama secara langsung memakai event `input`.
3. Ganti isi daftar kopi dengan hanya produk **robusta** memakai `filter`.
4. Pada proyek Kopi Nusantara Versi 1, ganti angka tahun di footer dengan tahun otomatis: beri `<span id="tahun"></span>` di footer, lalu di JavaScript tulis `document.getElementById("tahun").textContent = new Date().getFullYear();`.

Rangkuman KB 2.3

- DOM mewakili halaman sebagai pohon objek; JavaScript memilih elemen (`getElementById`, `querySelector`) dan mengubahnya (`textContent`, `classList`).
- Event ditanggapi dengan `addEventListener`; `e.preventDefault()` mencegah perilaku bawaan seperti muat ulang formulir.
- `innerHTML` menafsirkan HTML, sehingga data pengguna harus dibersihkan (XSS).
- Tiga jenis kesalahan: syntax, runtime, dan logic. Console dan `console.log` adalah alat utama debugging.

Cek Kemampuan KB 2.3

- ☐ Saya dapat memilih elemen dan mengubah isinya dengan JavaScript.
- ☐ Saya dapat menulis event `click` dan membaca nilai dari kolom input.
- ☐ Saya dapat menjelaskan perbedaan `textContent` dan `innerHTML`.

- ☐ Saya dapat menyebutkan tiga jenis kesalahan dan memberi contohnya.
- ☐ Saya menemukan dan memperbaiki lima kode bermasalah pada Praktik 2.

Kesalahan umum: salah ketik `id` (huruf besar-kecil) sehingga elemen bernilai `null`; menaruh `<script>` sebelum elemen yang dipilih; lupa `.value` saat membaca input; lupa `.join("")` setelah `map`; lupa `e.preventDefault()` pada `submit`.